// PYTHON · PANDAS · SCIKIT-LEARN · MATPLOTLIB / SEABORN

Machine Learning Cheatsheet

A single reference from your first missing-value fix to a deployed, publicly shareable model — with a real-world example behind every concept, not just a name.

`beginner -> advanced` `print-ready A4 / Letter` `30+ pages`

01
Data Foundations
Cleaning, encoding, scaling, and seven chart types for exploration.

02
Core ML Concepts
Vocabulary, ten algorithms, and the metrics that judge them.

03
Deployment
Save, wrap, and ship a model to a live public URL.

Includes 3 complete portfolio projects & a one-page quick reference

python 3 · scikit-learn

Table of Contents

MACHINE LEARNING CHEATSHEET -- BEGINNER TO ADVANCED

How the level tags work

Every concept in this guide carries a level badge: ●○○ BEGINNER ●●○ INTERMEDIATE ●●● ADVANCED — the filled dots show progression (1 of 3, 2 of 3, 3 of 3), so the system still reads correctly if printed in grayscale, even without color.

Default stack

Python + pandas + scikit-learn + matplotlib / seaborn throughout, unless a step names another tool.

Every concept

Definition, when/how to use it, and a real-world application -- not just a name and a formula.

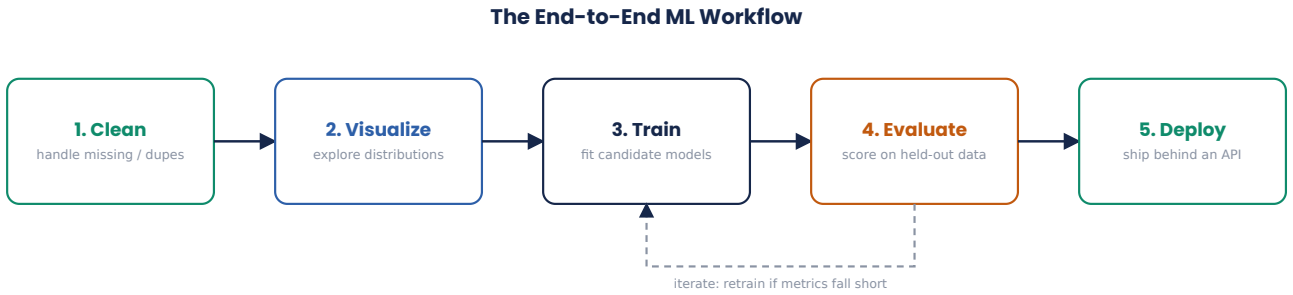
Print-ready

A4 / Letter-safe, ~12 mm margins, no table or code block split across a page break.

How This Guide Flows

THE SAME FIVE STEPS, EVERY TIME

Nearly every real machine learning project -- from a weekend Kaggle notebook to a production fraud model -- moves through the same five stages. This guide is organized to match: Section 1 covers the first two stages, Section 2 covers training and evaluation, and Section 3 covers the last stage, shipping a model somewhere other people can use it.



Clean data first, or every downstream chart and model inherits its errors. Evaluate before you deploy, or you ship blind.

Section 1 · Foundations

●○○ BEGINNER

Cleaning messy real-world data and visualizing it so patterns are visible before any model is trained.

Section 2 · Core ML

●●○ INTERMEDIATE

The vocabulary, the ten algorithms worth knowing cold, and how to score whether a model is any good.

Section 3 · Deployment

●●● ADVANCED

Packaging a trained model and putting it behind a real, shareable URL -- safely.

Section 4 · Portfolio Projects

Three complete, increasingly difficult projects that put every earlier section to work end-to-end -- from a first classifier to a deployed, publicly demoable app. Section 5, on the very last page, condenses the whole guide onto one page for fast lookup.

One-Time Environment Setup

```
pip install pandas scikit-learn matplotlib seaborn joblib
# add for Section 3 (deployment):
pip install streamlit fastapi uvicorn skl2onnx onnxruntime
```

01 Data Foundations

BEGINNER · CLEANING AND SEEING YOUR DATA BEFORE ANYTHING ELSE

Data Cleaning

Five checks to run, roughly in this order, on any new dataset before it touches a model.

Missing Values

●○○ BEGINNER

Entries in a dataset where no value was recorded (NaN / null). Left unhandled, they crash most scikit-learn estimators or silently bias results.

When to use it: Always check first, before any modeling. Use imputation when data is missing at random and rows are valuable; drop rows/columns when missingness is extreme (>50%) or random enough not to bias the sample.

```
import pandas as pd
from sklearn.impute import SimpleImputer

df = pd.read_csv("customers.csv")
print(df.isna().sum())          # audit missingness per column

num_cols = df.select_dtypes("number").columns
imputer = SimpleImputer(strategy="median")
df[num_cols] = imputer.fit_transform(df[num_cols])
df = df.dropna(subset=["signup_date"])   # drop rows missing a key field
```

Real-world use: A telecom churn model: usage minutes are missing for customers on unlimited plans. Median-imputing preserves the row instead of discarding paying customers from the training set.

Duplicates

●○○ BEGINNER

Rows that are exact or near-exact repeats of another row, often from repeated form submissions, merges, or logging retries.

When to use it: Run right after loading any new dataset, and again after joins/merges, which commonly fan out duplicate rows.

```
import pandas as pd

df = pd.read_csv("orders.csv")
print(f"exact duplicates: {df.duplicated().sum()}")
df = df.drop_duplicates()

# duplicates on a business key (ignoring a timestamp column)
key_dupes = df.duplicated(subset=["order_id"], keep="first")
df = df[~key_dupes]
```

Real-world use: An e-commerce order log where a flaky checkout API retried a request 3 times: deduping on order_id prevents revenue from being triple-counted in a forecasting model.

Outliers

●○○ BEGINNER

Data points far outside the typical range for a feature. May be genuine rare events or data-entry errors -- the fix depends on which.

When to use it: Use the IQR or z-score rule for roughly-normal features; visualize with a box plot first (see Section 1) to confirm before removing anything.

```
import pandas as pd

q1, q3 = df["income"].quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr

is_outlier = (df["income"] < lower) | (df["income"] > upper)
print(f"flagged: {is_outlier.sum()} rows")

df_capped = df.copy()
df_capped["income"] = df["income"].clip(lower, upper)  # winsorize, keep the row
```

Real-world use: A credit-risk model where a handful of incomes were entered as 9,999,999 (a placeholder value). Capping instead of deleting keeps the applicant in the dataset without letting the typo dominate the fit.

Categorical Encoding

●○○ BEGINNER

Converting text/category columns into numbers models can use: one-hot for unordered categories, ordinal for ranked ones, target encoding for high-cardinality columns.

When to use it: One-hot for low-cardinality nominal features (e.g. color); ordinal encoding when order matters (e.g. size S/M/L); target/frequency encoding when a column has hundreds of categories.

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

df = pd.get_dummies(df, columns=["country"], drop_first=True)  # one-hot

size_order = [["S", "M", "L", "XL"]]
enc = OrdinalEncoder(categories=size_order)
df["size_rank"] = enc.fit_transform(df[["size"]])
```

Real-world use: A house-price model: 'neighborhood' (no order) is one-hot encoded, while 'condition' (Poor < Fair < Good < Excellent) is ordinal encoded so the model can learn the ranking directly.

Feature Scaling

●○○ BEGINNER

Rescaling numeric features to a common range (standardization to mean 0 / std 1, or min-max to [0,1]) so no feature dominates purely due to units.

When to use it: Required for distance- and gradient-based models: KNN, SVM, logistic regression, neural nets, k-means. Tree-based models (random forest, gradient boosting) don't need it.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # fit only on train
X_test_scaled = scaler.transform(X_test)         # reuse train's stats
```

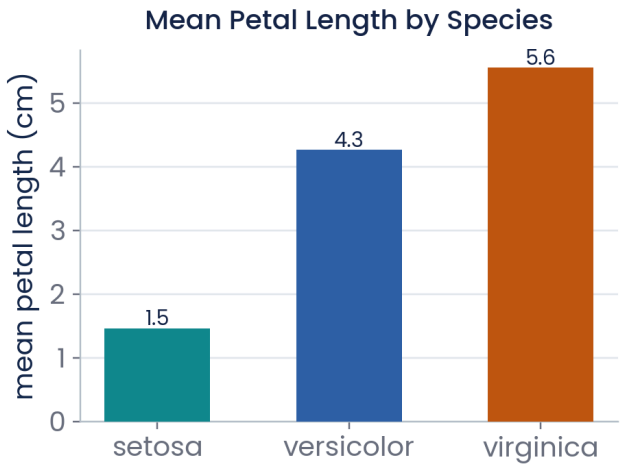
Real-world use: A KNN-based loan default model with 'age' (18-90) and 'income' ($20k-$500k): without scaling, income differences alone would dominate every distance calculation and age would be ignored.

01 Data Foundations

BEGINNER · CLEANING AND SEEING YOUR DATA BEFORE ANYTHING ELSE

Data Visualization

Seven chart types that cover most exploratory data analysis. Each thumbnail is rendered from the same sample dataset (the classic iris flower measurements) so you can compare how each chart type presents the same underlying data.

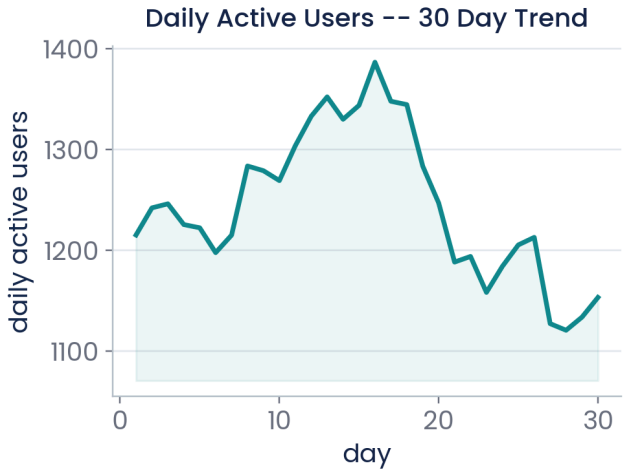


Bar Chart

Compares a numeric value across discrete categories using rectangular bars.

Best use case: Best for comparing a metric (mean, count, total) across a small number of groups.

```
means = df.groupby("species")["petal_length"].mean()
means.plot(kind="bar", color=
["#0F8B6B","#2D5FA8","#C1590F"])
plt.ylabel("mean petal length (cm)")
plt.title("Mean Petal Length by Species")
```

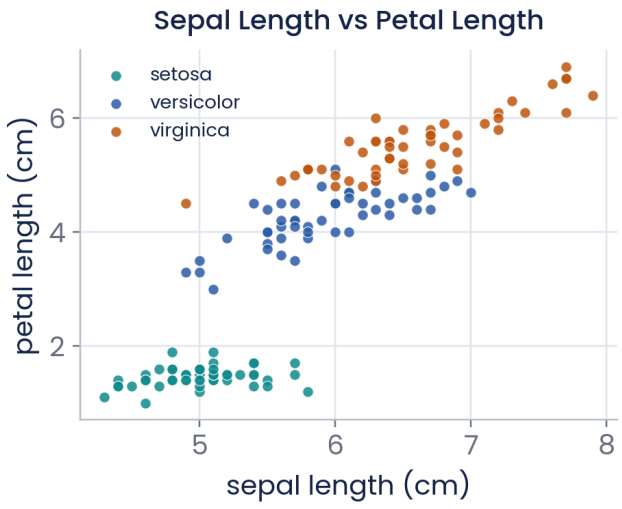


Line Chart

Plots values over a continuous sequence (usually time), connecting points to reveal trend and direction.

Best use case: Best for time series: revenue over months, sensor readings, model loss over training epochs.

```
df.plot(x="day", y="daily_active_users",
        kind="line", color="#0F8B6B", figsize=(6,4))
plt.ylabel("daily active users")
plt.title("Daily Active Users -- 30 Day Trend")
```

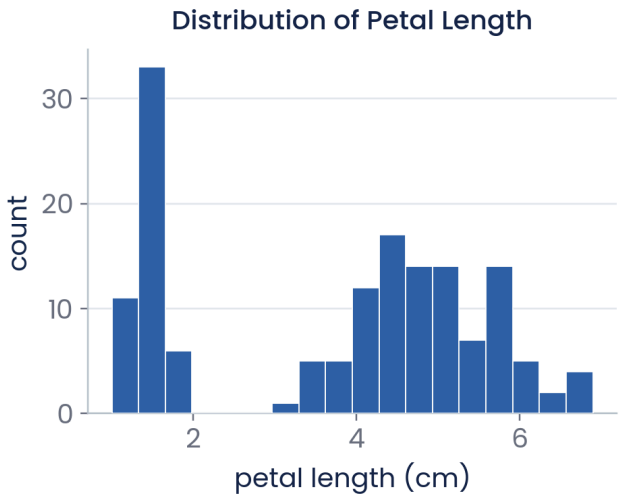


Scatter Plot

Plots two numeric variables as points, one per observation, to reveal correlation, clusters, or outliers.

Best use case: Best for inspecting the relationship between two continuous features before modeling.

```
import seaborn as sns
sns.scatterplot(data=df, x="sepal_length",
y="petal_length",
                hue="species", palette=
["#0F8B6B","#2D5FA8","#C1590F"])
plt.title("Sepal Length vs Petal Length")
```



Histogram

Bins a single numeric variable and shows the count of observations per bin, revealing the shape of its distribution.

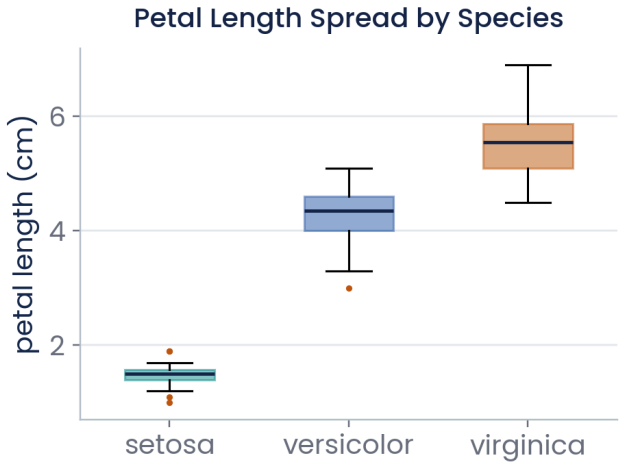
Best use case: Best for checking skew, spread, and modality of one feature before deciding on scaling or transforms.

```
df["petal_length"].plot(kind="hist", bins=18,
                        color="#2D5FA8",
edgecolor="white")
plt.xlabel("petal length (cm)")
plt.title("Distribution of Petal Length")
```

01 Data Foundations

BEGINNER · CLEANING AND SEEING YOUR DATA BEFORE ANYTHING ELSE

Data Visualization (continued)

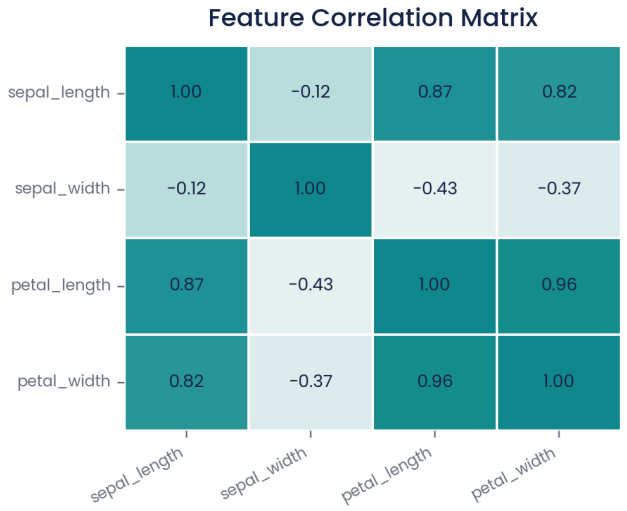


Box Plot

Summarizes a distribution's median, quartiles, and outliers (as individual points) per group, using the IQR rule.

Best use case: Best for comparing spread across groups and spotting outliers at a glance -- pairs naturally with the outlier-handling step above.

```
import seaborn as sns
sns.boxplot(data=df, x="species", y="petal_length",
            palette=["#0F8B6B","#2D5FA8","#C1590F"])
plt.title("Petal Length Spread by Species")
```

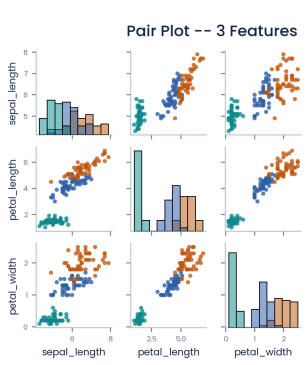


Heatmap

Encodes a matrix of values (often correlations) as color intensity in a grid, with numbers overlaid.

Best use case: Best for scanning a full correlation matrix at once to spot multicollinearity before feature selection.

```
import seaborn as sns
corr = df[num_cols].corr()
sns.heatmap(corr, annot=True, fmt=".2f",
cmap="viridis")
plt.title("Feature Correlation Matrix")
```



Pair Plot

A grid of scatter plots for every pair of numeric features, with histograms on the diagonal -- a fast first look at an entire dataset.

Best use case: Best as the very first exploratory step on a new dataset with a handful of numeric features and a class label to color by.

```
import seaborn as sns
sns.pairplot(df[["sepal_length","petal_length",
                 "petal_width","species"]],
             hue="species", diag_kind="hist")
```

A practical exploration order

On a brand-new dataset: pair plot first for a full overview, then histograms per feature, then a correlation heatmap before picking which features to engineer or drop.

02 Core ML Concepts

INTERMEDIATE · THE VOCABULARY EVERY ML CONVERSATION ASSUMES YOU KNOW

Foundational ML Definitions

The three learning paradigms, side by side.

Supervised Learning

●●○ INTERMEDIATE

Learning a mapping from inputs X to known output labels y, using examples where the correct answer is already provided.

When / how to use it: Use when you have historical, labeled examples of exactly what you want to predict.

Real-world use: Predicting whether a credit card transaction is fraudulent, using millions of past transactions already labeled fraud / not-fraud.

Unsupervised Learning

●●○ INTERMEDIATE

Finding structure (groups, patterns, compressed representations) in data that has no labels at all.

When / how to use it: Use for exploration, segmentation, or dimensionality reduction when you don't have -- or don't yet trust -- a target label.

Real-world use: Segmenting retail customers into behavioral groups (e.g. bargain hunters vs. loyal high-spenders) purely from purchase history, with no predefined labels.

Reinforcement Learning

●●○ INTERMEDIATE

An agent learns by taking actions in an environment and receiving rewards or penalties, optimizing for long-term cumulative reward rather than a single labeled answer.

When / how to use it: Use for sequential decision problems where actions affect future state -- games, robotics, resource allocation, recommendation ordering.

Real-world use: A warehouse robot that learns an efficient picking route by trial and error, getting a small reward for each completed pick and a penalty for collisions.

Overfitting & Underfitting

Underfitting

●○○ BEGINNER

The model is too simple to capture the underlying pattern -- poor performance on both training and test data.

When / how to use it: Fix by adding features, increasing model complexity, or reducing regularization strength.

Real-world use: A straight line fit to clearly seasonal retail sales data misses every peak and trough.

Overfitting

●○○ BEGINNER

The model memorizes training data, including its noise -- great training accuracy but poor generalization to new data.

When / how to use it: Fix with more training data, regularization, simpler models, dropout (neural nets), or early stopping.

Real-world use: A decision tree with unlimited depth that gets 99.9% training accuracy on patient records but only 61% on new patients.

02 Core ML Concepts

INTERMEDIATE · BIAS, VARIANCE, AND HOW MODELS ACTUALLY LEARN

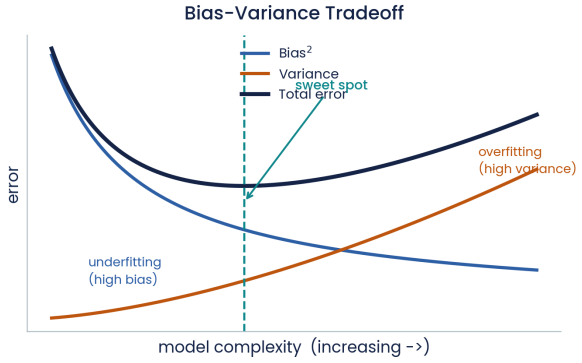
Bias-Variance Tradeoff

●●○ INTERMEDIATE

Bias is error from overly simple assumptions (the model can't capture the true pattern). Variance is error from being overly sensitive to the training data's noise. Total error = bias^2 + variance + irreducible noise, and the two typically trade off as model complexity changes.

When / how to use it: Use this framework to diagnose *why* a model is underperforming, and whether the fix is a more flexible model (reduce bias) or more regularization / data (reduce variance).

Real-world use: A linear model underfitting a clearly curved sales-vs-price relationship (high bias) vs. a 20-depth decision tree that memorizes training noise and collapses on new data (high variance).



As model complexity rises, bias falls and variance rises -- total error is minimized somewhere in the middle.

Train / Validation / Test Split

●○○ BEGINNER

Splitting data into three disjoint sets: train (fit the model), validation (tune hyperparameters and compare models), test (final, untouched check of real-world performance).

When / how to use it: Always split before touching the modeling step. A common ratio is 70/15/15 or 80/10/10; for small datasets, use cross-validation instead of a separate validation set.

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42,
stratify=y_temp)
# -> 70% train, 15% val, 15% test
```

Real-world use: A hospital readmission model: the test set is held out entirely until the very end, so the reported accuracy reflects performance on patients the model has truly never seen.

Cross-Validation

●●○ INTERMEDIATE

Splitting the training data into k folds, training on k-1 folds and validating on the remaining one, k times -- then averaging the scores for a more reliable performance estimate than a single split.

When / how to use it: Use when data is limited and a single validation split would be noisy, or when comparing several models/hyperparameter settings fairly.

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

scores =
cross_val_score(RandomForestClassifier(random_state=4
2),
                                X_train, y_train, cv=5,
scoring="f1")
print(f"F1: {scores.mean():.3f} +/-
{scores.std():.3f}")
```

Real-world use: A medical diagnosis dataset with only 400 patients: 5-fold CV gives a far more trustworthy accuracy estimate than one lucky (or unlucky) 80/20 split.

02 Core ML Concepts

INTERMEDIATE · OPTIMIZATION AND CONTROLLING MODEL COMPLEXITY

Gradient Descent

●●○ INTERMEDIATE

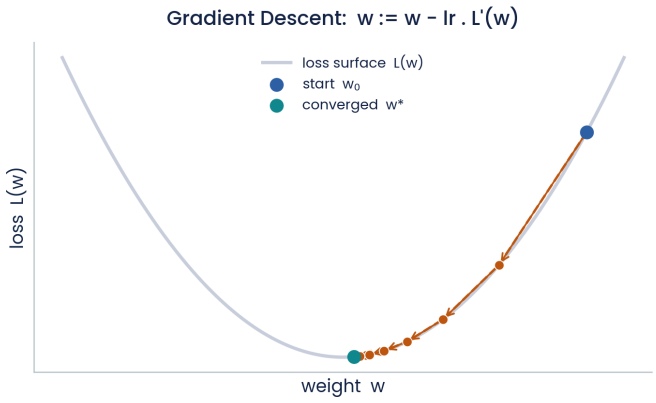
An iterative optimization algorithm that updates model parameters in the direction that most reduces the loss function, scaled by a learning rate.

```
w := w - learning_rate * dL/dw
```

When / how to use it: The core training mechanism behind linear/logistic regression, neural networks, and gradient boosting. Learning rate too high -> diverges; too low -> painfully slow.

```python
import numpy as np
w, lr = 5.2, 0.18               # start point + learning rate
for step in range(7):
    grad = 2 * w                # dL/dw for L(w) = w^2 + 3
    w = w - lr * grad           # move against the gradient
    print(f"step {step}: w={w:.3f}  loss={w**2+3:.3f}")
```

Real-world use: Training a logistic regression fraud classifier: each gradient step nudges every feature's weight slightly to reduce prediction error across the whole training batch.



Each step moves the weight against the gradient of the loss, shrinking the step as the surface flattens near the minimum.

Regularization

●●○ INTERMEDIATE

Adding a penalty term for model complexity (L1/Lasso shrinks some weights to exactly zero; L2/Ridge shrinks all weights smoothly) to reduce overfitting.

When / how to use it: Use when a model has many features relative to observations, or shows a large gap between training and validation scores.

```python
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0).fit(X_train, y_train)     # L2: shrinks weights
lasso = Lasso(alpha=0.1).fit(X_train, y_train)      # L1: can zero-out weights
print("features kept by Lasso:", (lasso.coef_ != 0).sum())
```

Real-world use: A genomics model with 20,000 gene-expression features but only 300 patients: Lasso automatically zeroes out irrelevant genes, leaving an interpretable shortlist.

02 Core ML Concepts

INTERMEDIATE · TEN ALGORITHMS WORTH KNOWING COLD

Algorithm Comparison Table

Every model below follows the same scikit-learn pattern: instantiate with hyperparameters, `.fit(X, y)`, then `.predict(X)`. "Both" means the algorithm has a classifier and a regressor version in scikit-learn.

Category	Algorithm	One-line intuition	Best use case	Key hyperparameters	Commented code

Regression	**Linear Regression**	Fits the straight line (or hyperplane) that minimizes squared error.	Predicting a continuous number from features with a roughly linear relationship.	fit_intercept, none really needed -- add Ridge/Lasso for regularization	`from sklearn.linear_model import LinearRegression`
					`model = LinearRegression().fit(X_train, y_train)  # closed-form fit`
					`preds = model.predict(X_test)`

Classification	**Logistic Regression**	Linear regression passed through a sigmoid to output a class probability.	Fast, interpretable baseline for binary/multiclass classification.	C (inverse regularization strength), penalty ('l1'/'l2')	`from sklearn.linear_model import LogisticRegression`
					`model = LogisticRegression(C=1.0, max_iter=1000)`
					`model.fit(X_train, y_train)`
					`proba = model.predict_proba(X_test)[:, 1]`
					`# P(class=1)`

Both	**Decision Tree**	Learns a series of if/else splits on features that best separate the target.	When you need a human-readable set of rules explaining each prediction.	max_depth, min_samples_leaf, criterion	`from sklearn.tree import DecisionTreeClassifier`
					`model = DecisionTreeClassifier(max_depth=5, min_samples_leaf=10)`
					`model.fit(X_train, y_train)`
					`# prune via max_depth`

Both	**Random Forest**	Averages many decision trees, each trained on a bootstrapped sample of rows and features.	Strong, low-maintenance default for tabular data; resists overfitting better than a single tree.	n_estimators, max_depth, max_features	`from sklearn.ensemble import RandomForestClassifier`
					`model = RandomForestClassifier(n_estimators=300, max_depth=None,`
					`random_state=42, n_jobs=-1)`
					`model.fit(X_train, y_train)`

Both	**SVM**	Finds the hyperplane that maximizes the margin between classes (or fits within a margin, for regression).	Small-to-medium, high-dimensional data (e.g. text) with a clear margin between classes.	C (margin softness), kernel ('linear'/'rbf'), gamma	`from sklearn.svm import SVC`
					`model = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True)`
					`model.fit(X_train_scaled, y_train)  # needs scaled features`

Both	**K-Nearest Neighbors**	Predicts using the majority label (or average value) of the k closest training points.	Simple baseline when local similarity is meaningful and the dataset is small.	n_neighbors, weights ('uniform'/'distance'), metric	`from sklearn.neighbors import KNeighborsClassifier`
					`model = KNeighborsClassifier(n_neighbors=7, weights="distance")`
					`model.fit(X_train_scaled, y_train)  # needs scaled features`

Clustering	**K-Means**	Partitions unlabeled data into k clusters by repeatedly assigning points to the nearest centroid and recomputing centroids.	Customer/behavioral segmentation, image color quantization, when there's no target label.	n_clusters (k), n_init, init method	`from sklearn.cluster import KMeans`
					`model = KMeans(n_clusters=4, n_init=10, random_state=42)`
					`labels = model.fit_predict(X_scaled)`
					`# cluster id per row`

Classification	**Naive Bayes**	Applies Bayes' theorem assuming features are conditionally independent given the class.	Text classification (spam, sentiment) with word-count / TF-IDF features -- fast and works well with little data.	alpha (Laplace smoothing)	`from sklearn.naive_bayes import MultinomialNB`
					`model = MultinomialNB(alpha=1.0)`
					`model.fit(X_train_counts, y_train)  # e.g. TF-IDF input`

Category	Algorithm	One-line intuition	Best use case	Key hyperparameters	Commented code

Both	**Gradient Boosting**	Builds trees sequentially, each one correcting the errors (residuals) of the ones before it.	Top accuracy on structured/tabular data -- the go-to for competitions and production tabular models.	n_estimators, learning_rate, max_depth	`from sklearn.ensemble import GradientBoostingClassifier`
					`model = GradientBoostingClassifier(n_estimators=300,`
					`learning_rate=0.05, max_depth=3, random_state=42)`
					`model.fit(X_train, y_train)`

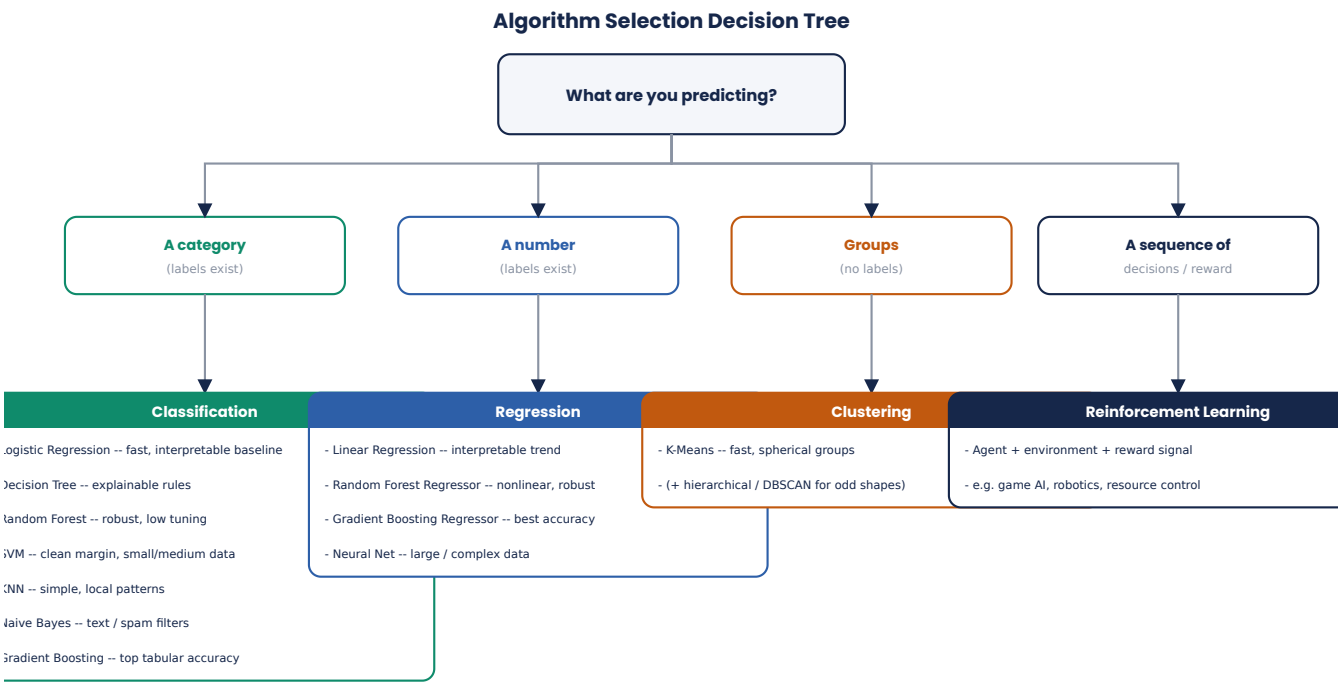
Both	**Basic Neural Net**	Layers of weighted sums + nonlinear activations, trained end-to-end with gradient descent (backpropagation).	Large datasets, unstructured data (images/text/audio), or complex nonlinear patterns tabular models can't capture.	hidden_layer_sizes, learning_rate_init, activation	`from sklearn.neural_network import MLPClassifier`
					`model = MLPClassifier(hidden_layer_sizes=(64, 32),`
					`activation="relu", max_iter=500, random_state=42)`
					`model.fit(X_train_scaled, y_train)`

02 Core ML Concepts

INTERMEDIATE · WHICH ALGORITHM SHOULD I EVEN TRY FIRST?

Algorithm-Selection Decision Tree

A practical starting point, not a strict rule -- always confirm the choice with cross-validation (Section 2.1).



Rule of thumb

Start simple (linear/logistic regression), confirm the baseline works end-to-end, *then* reach for random forest or gradient boosting for accuracy. Only reach for a neural net when the data is large and unstructured (images, text, audio), or simpler models have clearly plateaued.

02 Core ML Concepts

INTERMEDIATE · SCORING WHETHER A MODEL IS ACTUALLY GOOD

Evaluation Metrics

The first four are for classification, the last three for regression. ROC-AUC needs predicted probabilities (`predict_proba`), not hard class labels.

Metric	Type	Definition	Formula	When to use	Code
Accuracy	classification	Share of all predictions that were correct.	`(TP + TN) / (TP + TN + FP + FN)`	Use only when classes are roughly balanced -- misleading on imbalanced data (e.g. 99% non-fraud).	`from sklearn.metrics import accuracy_score` `accuracy_score(y_test, y_pred)`
Precision	classification	Of everything predicted positive, the share that was actually positive.	`TP / (TP + FP)`	Use when false positives are costly -- e.g. flagging a legitimate transaction as fraud.	`from sklearn.metrics import precision_score` `precision_score(y_test, y_pred)`
Recall	classification	Of all actual positives, the share the model correctly found.	`TP / (TP + FN)`	Use when false negatives are costly -- e.g. missing an actual cancer diagnosis.	`from sklearn.metrics import recall_score` `recall_score(y_test, y_pred)`
F1 Score	classification	Harmonic mean of precision and recall -- balances both into one number.	`2 * (precision * recall) / (precision + recall)`	Use as a single summary metric on imbalanced data when both false positives and negatives matter.	`from sklearn.metrics import f1_score` `f1_score(y_test, y_pred)`
ROC-AUC	classification	Probability the model ranks a random positive example above a random negative one, across all thresholds.	`area under the True-Positive-Rate vs False-Positive-Rate curve`	Use to compare classifiers' ranking quality independent of a chosen decision threshold.	`from sklearn.metrics import roc_auc_score` `roc_auc_score(y_test, y_proba)  # needs probabilities, not labels`
RMSE	regression	Root Mean Squared Error -- average prediction error, in the same units as the target, penalizing large errors more.	`sqrt( mean( (y_true - y_pred)^2 ) )`	Use as the default regression metric when large errors should be penalized heavily (e.g. price prediction).	`from sklearn.metrics import root_mean_squared_error` `root_mean_squared_error(y_test, y_pred)`
MAE	regression	Mean Absolute Error -- average absolute prediction error, treating all errors linearly.	`mean( |y_true - y_pred| )`	Use when outliers shouldn't dominate the error signal, or for an easily-explainable metric.	`from sklearn.metrics import mean_absolute_error` `mean_absolute_error(y_test, y_pred)`
R-squared	regression	Proportion of variance in the target explained by the model, relative to a mean-only baseline.	`1 - SS_residual / SS_total`	Use to communicate overall model fit on a 0-1 (or negative, if worse than the mean) scale.	`from sklearn.metrics import r2_score` `r2_score(y_test, y_pred)`

Accuracy is not enough on imbalanced data

On a dataset that's 99% "not fraud," a model that always predicts "not fraud" scores 99% accuracy while catching zero fraud. Precision, recall, F1, and ROC-AUC exist precisely to catch this failure mode.

02 Core ML Concepts

INTERMEDIATE · TERMS THAT GET MIXED UP CONSTANTLY

Glossary of Frequently Confused Terms

Quick disambiguation for terms that sound similar but mean different things -- worth a skim before Section 3.

Parameter

A value the model learns from data during training (e.g. a regression coefficient, a tree split threshold).

Hyperparameter

A setting chosen before training that controls how learning happens (e.g. n_estimators, learning_rate) -- tuned via cross-validation, not learned.

Feature / Label

A feature (X) is an input column used to predict; the label (y) is the target value being predicted.

Epoch

One full pass through the entire training dataset -- neural nets typically train for many epochs.

Baseline Model

The simplest reasonable model (e.g. predict the mean, or the majority class) used as a sanity-check floor for any fancier model.

Confusion Matrix

A table of predicted vs actual classes (TP, TN, FP, FN) that precision, recall, and F1 are all computed from.

Class Imbalance

When one class vastly outnumbers another (e.g. 99% vs 1%) -- requires metrics beyond accuracy and often resampling.

Pipeline

A chained sequence of preprocessing + model steps (sklearn.pipeline.Pipeline) that fits and transforms consistently on train and test data.

Inference

Using an already-trained model to make a prediction on new data -- as opposed to training, which fits the model.

Latency vs Throughput

Latency is the time for one prediction to return; throughput is how many predictions can be served per second -- both matter once deployed.

03 Deployment

ADVANCED · PACKAGING A TRAINED MODEL FOR REUSE

Save & Load a Trained Model

A trained model only has value if you can reload it later without retraining. Three common formats, in order of how portable they are:

pickle

Python's native serializer. Simplest option, but not safe to load untrusted files and not portable outside Python.

```
import pickle
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

with open("model.pkl", "rb") as f:
    model = pickle.load(f)
```

joblib

Like pickle but far more efficient for objects with large NumPy arrays (most scikit-learn models). The standard choice for sklearn.

```
import joblib
joblib.dump(model, "model.joblib")   # save
model = joblib.load("model.joblib")  # load
```

ONNX

An open, cross-framework format. Slightly more setup, but the model can then run in Java, C++, JavaScript, or mobile runtimes -- not just Python.

```
from skl2onnx import to_onnx
onx = to_onnx(model, X_train[:1].astype("float32"))
with open("model.onnx", "wb") as f:
    f.write(onx.SerializeToString())

# inference anywhere, without scikit-learn installed:
import onnxruntime as rt
sess = rt.InferenceSession("model.onnx")
pred = sess.run(None, {"X": X_test.astype("float32")})
```

03 Deployment

ADVANCED · WRAPPING A MODEL BEHIND AN INTERFACE

Build an API or App

Two common wrappers for the same saved model — a JSON API for other programs to call (FastAPI), or an interactive demo people can click through in a browser (Streamlit). Both load the model once at startup, not on every request.

Option A — FastAPI: programmatic access

Best when another program (a website, a mobile app, another service) needs to call your model over HTTP and get JSON back.

```
# app.py -- a small, fully commented prediction API
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI(title="Iris Classifier API")
model = joblib.load("model.joblib")          # loaded once at startup

class Features(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.post("/predict")
def predict(features: Features):
    X = np.array([[features.sepal_length, features.sepal_width,
                   features.petal_length, features.petal_width]])
    pred = model.predict(X)[0]
    proba = model.predict_proba(X).max()
    return {"prediction": str(pred), "confidence": round(float(proba), 3)}

@app.get("/health")
def health():
    return {"status": "ok"}                  # for uptime checks

# run locally with:  uvicorn app:app --reload
```

Try it once it's running

Visit `http://localhost:8000/docs` for an auto-generated, interactive API explorer — FastAPI builds this for free from the Pydantic model above.

03 Deployment

ADVANCED · WRAPPING A MODEL BEHIND AN INTERFACE

Option B — Streamlit: interactive demo

Best when a human should be able to try the model directly in a browser, with no separate frontend to build. The deployment walkthrough on the next page ships exactly this app.

```
# streamlit_app.py -- a small, fully commented interactive demo
import streamlit as st
import joblib
import numpy as np

st.set_page_config(page_title="Iris Classifier", page_icon=":herb:")
model = joblib.load("model.joblib")           # loaded once, cached by Streamlit

st.title("Iris Species Classifier")
st.write("Move the sliders to describe a flower and get a live prediction.")

# sliders become the model's input features
sepal_length = st.slider("Sepal length (cm)", 4.0, 8.0, 5.8)
sepal_width  = st.slider("Sepal width (cm)",  2.0, 4.5, 3.0)
petal_length = st.slider("Petal length (cm)", 1.0, 7.0, 4.0)
petal_width  = st.slider("Petal width (cm)",  0.1, 2.5, 1.2)

if st.button("Predict"):
    X = np.array([[sepal_length, sepal_width, petal_length, petal_width]])
    pred = model.predict(X)[0]
    proba = model.predict_proba(X).max()
    st.success(f"Predicted species: **{pred}**  (confidence: {proba:.1%})")

# run locally with:  streamlit run streamlit_app.py
```

Which should I pick?

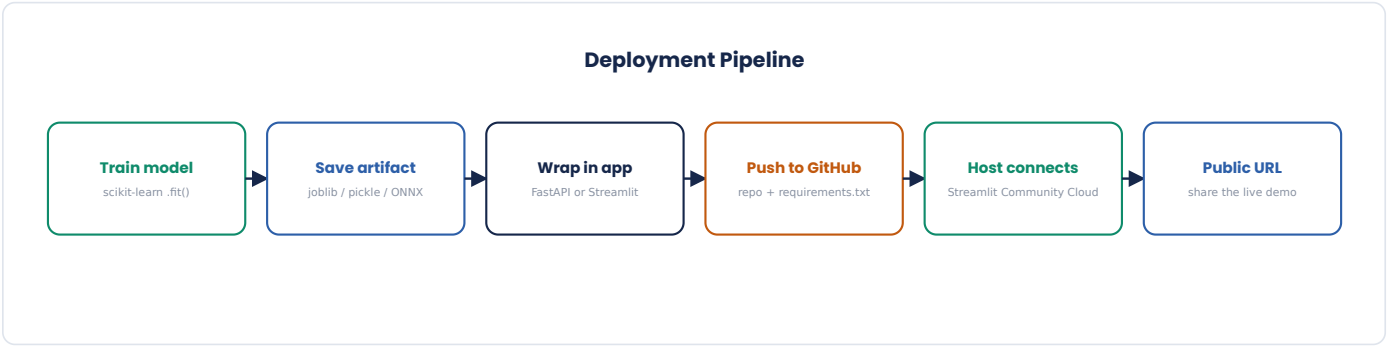
Building a demo for people to click around in? Streamlit. Building something another piece of software will call programmatically, or need to scale behind a load balancer? FastAPI. Many real projects ship both.

03 Deployment

ADVANCED · SHIPPING TO A LIVE, PUBLIC URL

Deploy to the Cloud

Streamlit Community Cloud is free, connects directly to GitHub, and needs no server management — the steps below take a repo to a public URL in a few minutes.



1. **Prep the repo**
   Put streamlit_app.py, model.joblib, and requirements.txt (streamlit, scikit-learn, joblib, numpy) in a public GitHub repository.
2. **Sign in**
   Go to share.streamlit.io and sign in with your GitHub account -- no separate password to manage.
3. **New app**
   Click 'New app', pick the repo, branch, and entry-point file (streamlit_app.py).
4. **Deploy**
   Click Deploy. Community Cloud builds a container from requirements.txt and installs your dependencies automatically.
5. **Get the URL**
   Your app is live at a shareable https://<your-app>.streamlit.app URL within a few minutes -- share it with anyone.
6. **Update**
   Push new commits to the connected branch and the live app redeploys automatically -- no manual redeploy step.

Free tier notes (verify current limits before relying on them)

Streamlit Community Cloud is free for public apps: roughly 1 GB RAM per app, apps sleep after long idle periods and wake on the next visit, and you get one private app at a time. Hugging Face Spaces and Render's free tier are solid alternatives with similar GitHub-connected workflows if you need more headroom.

03 Deployment

ADVANCED · KEEPING A PUBLIC DEMO SAFE AND STABLE

Security & Rate-Limiting for a Public Demo

A public URL means anyone can hit it — a few defensive basics before sharing the link widely.

- Never hardcode API keys, database URLs, or secrets in the repo -- use the platform's secrets manager (e.g. Streamlit's st.secrets) and add .env / secrets files to .gitignore.
- Validate and bound all user input server-side (Pydantic models in FastAPI do this for free) -- never trust a slider or text box to stay in range.
- Add basic rate-limiting on public endpoints (e.g. slowapi for FastAPI) so one visitor can't exhaust your free-tier compute with a request loop.
- Cache expensive operations (@st.cache_resource for the loaded model, @st.cache_data for data) so repeated visits don't reload or recompute unnecessarily.
- Log predictions and errors, but never log personally identifiable input data on a public free-tier host without a clear retention and privacy policy.
- Set a request size/timeout limit and return generic error messages -- detailed stack traces in a public response can leak implementation details.

The most common first mistake

Committing an API key or database password directly into a public GitHub repo. Rotate any credential the moment you realize it happened — assume it was scraped within minutes, since bots continuously scan public commits for exposed secrets.

04 Portfolio Projects

BEGINNER · PROJECT 1 OF 3

01 · Titanic Survival Classifier

●○○ BEGINNER

Predict whether a passenger survived the Titanic disaster from features like class, age, sex, and fare -- the canonical first classification project.

DATASET SOURCE

Kaggle: “Titanic - Machine Learning from Disaster” (train.csv, ~891 rows)

EXPECTED OUTPUT

Validation accuracy around 0.80-0.83 with logistic regression; a tuned random forest typically reaches ~0.83-0.86.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("train.csv")

# --- clean & engineer ---
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True)

features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
            "Embarked_Q", "Embarked_S"]
X, y = df[features], df["Survived"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# --- train & evaluate ---
model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
preds = model.predict(X_test)
print(f"accuracy: {accuracy_score(y_test, preds):.3f}")
print(confusion_matrix(y_test, preds))
```

COMMON PITFALL

Encoding Sex/Embarked before splitting train/test (leaks info); forgetting stratify=y on a class-imbalanced target.

TRY EXTENDING IT

Engineer a FamilySize = SibSp + Parch + 1 feature, then compare against a random forest baseline.

SKILL DEMONSTRATED

End-to-end classification workflow: cleaning, encoding, a baseline model, and evaluation with a confusion matrix.

04 Portfolio Projects

INTERMEDIATE · PROJECT 2 OF 3

02 · California House Price Regression

●●○ INTERMEDIATE

Predict median house value for a district from census features (income, rooms, location, population density).

DATASET SOURCE

Built into scikit-learn: sklearn.datasets.fetch_california_housing (20,640 rows, 8 numeric features)

EXPECTED OUTPUT

Gradient boosting typically reaches RMSE ≈ $45,000-50,000 and R^2 ≈ 0.82-0.85 on the held-out test set.

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import root_mean_squared_error, r2_score

data = fetch_california_housing(as_frame=True)
X, y = data.data, data.target * 100_000       # target is in $100k units

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

model = GradientBoostingRegressor(n_estimators=400, max_depth=3,
                                  learning_rate=0.05, random_state=42)
model.fit(X_train_s, y_train)

preds = model.predict(X_test_s)
print(f"RMSE: ${root_mean_squared_error(y_test, preds):,.0f}")
print(f"R2:   {r2_score(y_test, preds):.3f}")
```

COMMON PITFALL

Fitting the scaler on the full dataset instead of X_train only (leaks test statistics into training).

TRY EXTENDING IT

Add a GridSearchCV over max_depth and learning_rate, and compare RMSE against a plain RandomForestRegressor baseline.

SKILL DEMONSTRATED

Full regression pipeline with scaling, a gradient boosting model, and regression-specific metrics (RMSE, MAE, R^2) instead of classification ones.

04 Portfolio Projects

ADVANCED · PROJECT 3 OF 3

03 · Deployed Handwritten-Digit Classifier

●●● ADVANCED

Train an image classifier on handwritten digits and ship it as a live, public web demo people can draw on and test in the browser.

DATASET SOURCE

Built into scikit-learn: sklearn.datasets.load_digits (1,797 images, 8x8 grayscale, digits 0-9)

EXPECTED OUTPUT

A live URL such as https://digit-classifier.streamlit.app where a visitor draws a digit and instantly sees the predicted class and confidence.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report
import joblib

digits = load_digits()
X, y = digits.data, digits.target            # 64 pixel values per image

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = SVC(kernel="rbf", gamma=0.001, C=10, probability=True)
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))

joblib.dump(model, "digit_model.joblib")       # -> wrap in Streamlit,
                                               #    push to GitHub, deploy
                                               #    (see Section 3)
```

COMMON PITFALL

Forgetting probability=True on SVC means predict_proba isn't available for a confidence score in the app.

TRY EXTENDING IT

Add a canvas component (e.g. streamlit-drawable-canvas) so visitors draw a digit by hand instead of picking a test-set example.

SKILL DEMONSTRATED

The complete cycle from Section 3: train, save with joblib, wrap in a Streamlit app, and deploy to Streamlit Community Cloud with a public URL.

Quick Reference

ML CHEATSHEET · BACK COVER

One page, no paging through the rest — key formulas, the algorithm table, and the commands you reach for most.

KEY FORMULAS

Accuracy: `(TP+TN) / (TP+TN+FP+FN)`

Precision: `TP / (TP+FP)`

Recall: `TP / (TP+FN)`

F1 Score: `2PR / (P+R)`

RMSE: `sqrt(mean((y-y_hat)^2))`

MAE: `mean(|y-y_hat|)`

R-squared: `1 - SS_res/SS_tot`

Gradient step: `w := w - lr * dL/dw`

Bias-variance: `err = bias^2 + var + noise`

Standardize: `z = (x - mean) / std`

ALGORITHM CHEAT TABLE

ALGORITHM	TYPE	BEST FOR
Linear Reg.	regression	linear trend
Logistic Reg.	classif.	fast baseline
Decision Tree	both	explainable rules
Random Forest	both	robust default
SVM	both	margin, small data
KNN	both	local similarity
K-Means	clustering	no labels
Naive Bayes	classif.	text / spam
Grad. Boosting	both	best tabular acc.
Neural Net	both	images/text/complex

COMMON COMMANDS

`df.isna().sum()` - audit missing values

`df.drop_duplicates()` - remove duplicate rows

`df.fillna(df.median())` - impute missing

`pd.get_dummies(df, cols)` - one-hot encode

`StandardScaler().fit_transform(X)` - scale features

`train_test_split(X, y, test_size=.2)` - split data

`model.fit(X_train, y_train)` - train a model

`model.predict(X_test)` - get predictions

`cross_val_score(model, X, y, cv=5)` - k-fold CV

`joblib.dump(model, 'm.joblib')` - save model

`joblib.load('m.joblib')` - load model

`streamlit run app.py` - run app locally

`uvicorn app:app --reload` - run API locally

`git push origin main` - trigger redeploy

Data -> sklearn.model_selection -> fit -> sklearn.metrics -> joblib -> Streamlit / FastAPI -> live URL

Full guide: Sections 1-4